

Lab 2: Energy Efficient Displays

Your Name

Student ID

October 3, 2026

1 Introduction

2 Background

2.1 Spiking Neural Networks

Spiking Neural Networks (SNNs) are a class of artificial intelligence. Aiming to mimic how a brain works. They process information in discrete-time with spikes. These spikes are essentially ones and zeros for a neuron's output. This makes them great for edge devices with energy and performance constraints.

There are different neuron models; some are focused on biophysical accuracy, and some are focused on simplicity. The Leaky Integrate-and-Fire (LIF) models are, in a sense, in the middle.

2.1.1 Leaky Integrate And Fire Model

In the LIF neuron model, every neuron has an individual membrane potential that decays over time. Every time a neuron's membrane potential passes the specified threshold, a spike occurs, and its membrane potential is reset with a mechanism.

There are two mechanisms: reset-by-subtract (soft reset) and reset-to-zero (hard reset). The soft-reset mechanism works by subtracting the threshold when a spike occurs; on the other hand, with a hard-reset, it resets to zero.

The LIF model's membrane potential equation is given in Equation 1. This equation is being used to calculate the membrane potential of the next timestep. The membrane potential decays at every timestep with a specific decay rate; then, the accumulated input is summed, and lastly the soft-reset is applied.

$$U[t + 1] = \underbrace{\beta U[t]}_{\text{decay}} + \underbrace{W X[t + 1]}_{\text{input}} - \underbrace{S[t] U_{\text{thr}}}_{\text{reset}} \quad (1)$$

$U[t]$ Membrane potential at time step t .

β Membrane decay factor.

W Learnable parameter (weight).

$X[t + 1]$ Input voltage of the next time step.

$S[t]$ Spike indicator.

U_{thr} Thershold potential.

2.1.2 Reset Mechanisms

The reset mechanisms are reset-by-subtract (soft reset) and reset-to-zero (hard reset), as stated before.

Soft reset is when a spike occurs; it subtracts the threshold, which allows the residual potential to remain, providing more accuracy. With a hard reset, the accuracy might drop, but it might be cheaper to reset a register to zero [1].

2.2 Spiking Convolutional Neural Networks

Spiking Convolutional Neural Networks are SNNs with convolutional connections. One of the best networks that achieves 99.1% accuracy on the MNIST dataset is a Spiking ConvNet, which is trained as an ANN and then transformed into a Spiking ConvNet in Diehl et al. [2]. This work also has the fully connected network version, which has a 98.6% accuracy with the MNIST dataset. These results suggest that the convolutional connectivity can provide an advantage. Thus, making the convolutional support for the C code generation crucial.

2.2.1 Convolution

The convolution operation in neural networks occurs when a learnable kernel shifts over an input tensor. The 2-dimensional convolutional neural network has an input tensor of rank 2, with neurons represented in a 2-D array. They are connected to the feature map (output matrix) with learnable kernels. This introduces weight sharing, which reduces the memory load.

An example input feature map, kernel, and the output feature map are given in Figure 1. Every element of the input and output matrices represents a neuron (or activation). The convolution operation's steps are shown in Figure 2. The kernel's every element is multiplied by the corresponding element of the input matrix, and then they are summed up. Lastly, this value is saved to the corresponding position of the output neuron. This process is repeated until the input matrix is traversed.

It is understandable that the output matrix's size depends on the kernel, stride and padding parameters. The stride parameter decides how much kernel moves to the next position, if stride increases the output matrix dimension will be smaller. The padding parameter is also configurable, vary in size and values. It works with adding specific values to the borders to prevent the loss of border feature information. And both parameters control the dimensions of the output matrix. [?].

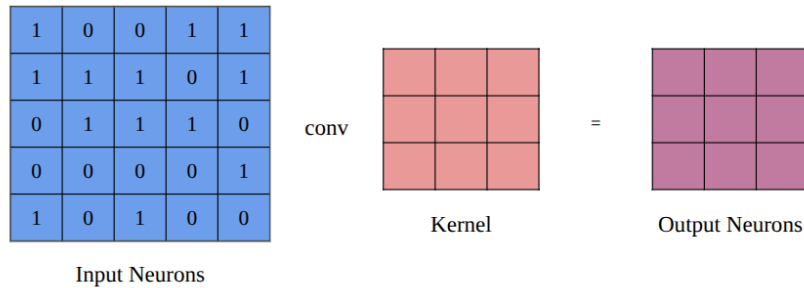


Figure 1: An example 2-dimensional convolutional layer

1	0	0	1	1
1	1	1	0	1
0	1	1	1	0
0	0	0	0	1
1	0	1	0	0

1	0	0	1	1
1	1	1	0	1
0	1	1	1	0
0	0	0	0	1
1	0	1	0	0

1	0	0	1	1
1	1	1	0	1
0	1	1	1	0
0	0	0	0	1
1	0	1	0	0

1	0	0	1	1
1	1	1	0	1
0	1	1	1	0
0	0	0	0	1
1	0	1	0	0

.....

Figure 2: Convolution Steps with padding none and stride one

Two dimensional convolution support is added within the thesis scope, as basic and optimized options. While basic convolution uses nested for loops to calculate the indexes and accumulates, optimized convolution uses a pre-computed connectivity table and accumulates only the active (spiked) inputs.

3 Related Work

TODO: Bu bi yere eklenebilir <https://arxiv.org/abs/2406.02528> Scalable MatMul-free Language Modeling Basically, they turn the weights into ternary (-1,0,1) This makes the huge LLM MAC operations to just accumulate ops.

In the area of *****, there are different approaches to optimize the SNN performance. Some of them focuses on the pre-processes like pruning, some exploits the sparse nature of SNN, others optimizes the inference with different methods.

In recent works of deployment SNNs on edge devices, Del Vecchio (2025) introduced `snn2mcu`, an open-source C code generator for STM32 microcontrollers with cortex-M7 CPU. To eliminate manual firmware generation, Del Vecchio created this work to automatically generate optimized C codes from Neuromorphic Intermediate Representation (NIR) model graphs. The generator uses optimized CMSIS-DSP and CMSIS-NN library functions to achieve an efficient inference. However, `snn2mcu` lacks the support for soft-reset mechanism and convolution connectivity. Building upon Del Vecchio’s work, this thesis extends it with soft-reset mechanism and event-driven convolutional support.

Several works targets the efficiency with weight sparsity and pruning. Lightweight Software Kernels and Hardware Extensions for Efficient Sparse Deep Neural Networks on Microcontrollers [3], uses N:M structured sparsity to store the weights, the indices of the weights are stored to be used in execution. They introduce decimated `im2col` buffer to be used with the weight indices to calculate the inference. This approach reduces the memory access count with the cost of not using optimized CMSIS-DSP library. Thus the trade-off point depends on the sparsity ratio. Another work, Compression and Inference of Spiking Neural Networks on Resource-Constrained Hardware [4], built a toolchain which prunes the near-zero weights, compresses the model and generates C representations.

On the other hand some works focuses on activation sparisty and execution optimization. Efficient CNN Inference on Ultra-Low-Power MCUs via Saturation-Aware Convolution [5], skips

the multiply and accumulate (MAC) operations for the already saturated neurons.

To achieve activation sparsity in convolutional networks, the effected output neuron needs to be known. There are couple of approaches, storing a pre-computed rulebook or calculating the address on the go, are among those.

Submanifold Sparse Convolutional Networks [6], constructs a rule-book with the active (non ground-state) neurons and its effective output neuron pairs, then this is used in GEMM operation. Moreover, the rule-book has only the output neurons which is their receptive field's center is active. Consequently, the features cannot dilate or spread, which results in a slight increase of the error rates.

Spike-based dynamic computing with asynchronous sensing-computing neuromorphic chip [7], uses a function to find the affected output to achieve asynchronous, event-driven convolution. When a spike occurs, its address mapping function outputs the affected address on the output map. Then, the kernel is placed at that input point as an anchor, and the convolution is calculated. This eliminates unnecessary calculations and memory, with the cost of overhead resulting from the address mapping.

One of the earliest examples of event-driven SNNs is Event-Driven Simulation of Networks of Spiking Neurons [8], instead of evaluating every neuron at every timestep, computation is performed only when a spike (an event) occurs. This mechanism works dynamically; every event and affected consecutive neuron is queued.

Add this one I am using a similar approach SpikeNET: An Event-driven Simulation Package for Modeling Large Networks of Spiking Neurons

<https://sccn.ucsd.edu/~arno/mypapers/delormeN2003.pdf>

From 2003

And maybe this one: They use look up tables for neuron and synapse dynamics to execute fast. From 2006 Event-Driven Simulation Scheme for Spiking Neural Networks Using Lookup Tables to Characterize Neuronal Dynamics

TODO: Explain different approaches for event-driven solutions

4 Methodology and Pipeline Overview

5 Optimized C Code Generation

The basis is the `nir_to_c_generator.py` script, which generates optimized C code for Cortex-M7 ARM MCUs. In the previous version, the capabilities include linear and recurrent connected neural networks with a hard-reset (Reset-to-value) mechanism. With my work, the soft-reset (reset-by-subtract) mechanism, convolutional (conv2d) support, and sparsity optimization were introduced.

TODO: Rephrase the sparsity part after implement it.

5.1 Soft Reset

The first thing to implement is Soft Reset (Reset-by-subtraction), The previous work had only the hard reset (Reset-to-value) and lacked the soft reset mechanism.

The soft-reset mechanism introduced here follows the same approach as in Eshraghian et al. [?]. Also, it is implemented in the `snnTorch` Python library in the same way, enabling the Python golden reference to match this work. The mathematical representation of the soft-reset mechanism is given in the last part of Equation 2.

$$U[t] = \underbrace{\beta U[t-1]}_{\text{decay}} + \underbrace{W X[t]}_{\text{input}} - \underbrace{S_{\text{out}}[t-1]\theta}_{\text{reset}} \quad (2)$$

In Figure 3, the soft-reset implementation in C is given. First, the current timestep membrane update is being calculated with vectorized ARM functions. After the membrane update, the spike check is done. While comparing the potential to the threshold, if there is a spike, the threshold value is saved to be subtracted at the next timestep.

```

// Membrane Update
q15_t temp1[num_neurons], temp2[num_neurons];

arm_mult_q15(membrane_potentials, decay_factors, temp1, num_neurons);
arm_add_q15(temp1, weighted_inputs, temp2, num_neurons);
arm_sub_q15(temp2, reset_values, membrane_potentials, num_neurons);

// Spike check, then store reset_value for the NEXT step
for (uint16_t i = 0; i < num_neurons; i++) {
    if (membrane_potentials[i] > thresholds[i]) {
        output_spikes[i] = 1;
        neurons[i].reset_value = thresholds[i]; // subtract next step
    } else {
        output_spikes[i] = 0;
        neurons[i].reset_value = 0;
    }
    neurons[i].membrane_potential = membrane_potentials[i];
}

```

Figure 3: Soft-Reset Implementation in C

5.2 Convolutional Support

The basic convolutional support's pseudo-code shown in the Figure 4.

The convolution calculation is done with a sliding window in a deductive manner, from the output neuron to the input neuron. For every output neuron an accumulator variable is set, then the where sliding window is pointing in the input feature map is calculated. With these coordinates the activation value and the weight is multiplied and added to the accumulation. This loop is repeated for all the output feature map.

5.3 Event-Driven Convolution

5.3.1 Compressed Sparse Row (CSR) Table

5.3.2 Implementation of CSR

5.3.3 Neuron Update with CSR

6 Embedded Deployment

In this work, Ubuntu 22.04 used as the development environment, github used as the version control and the STM32H750B-DK platform is used as a primary deployment environment.

TODO: Detail it more

6.0.1 STM32H750B-DK Discovery Kit

The STM32H750B-DK Discovery Kit, has an STM32H750xB Arm Cortex-M7 microcontroller. It has a LCD with a touch panel, two 512 Mbit Quad-SPI NOR flash memory, 128 Mbit SDRAM. The board's image is shown in Figure ??.

```

for each output channel o:
  for each output row h:
    for each output column w:
      accumulator = 0

      for each input channel i:
        for each kernel row fy:
          for each kernel column fx:

            ih = h * stride + fy - padding
            iw = w * stride + fx - padding

            if ih and iw in the bounds:
              accumulator = accumulator + input[i,ih,iw] * weight [o,i,fy,fx]

output[o,h,w] = accumulator

```

Figure 4: Basic Convolution

PUT THE BOARDS IMAGE.

The STM32H750xB is a 32-bit microcontroller built around the Arm Cortex-M7 core operating at clock frequencies up to 480 MHz. It has a 128 KBytes flash and a 1 Mbyte of RAM.

ST Microelectronics has several highly optimized Common Microcontroller Software Interface Standard (CMSIS) libraries for to be used by their microcontrollers. Such as CMSIS-DSP and CMSIS-NN, which are used in this work.

6.1 CMSIS-DSP and CMSIS-NN

The CMSIS-DSP [9], is a library for common compute processing functions. TODO: Explain
The CMSIS-NN [10] [11] TODO: Explain

6.2 UART DMA

6.3 Quad-SPI Flash Chips

6.4

7 Model Training and Testing

7.1 Braille

7.2 ST-MNIST

8 Experimental Results

9 Discussion

[?]

A Appendix

References

- [1] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, sep 2023.
- [2] Peter U. Diehl, Daniel Neil, Jonathan Binas, Matthew Cook, Shih-Chii Liu, and Michael Pfeiffer. Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2015.
- [3] Francesco Daghero, Daniele Jahier Pagliari, Francesco Conti, Luca Benini, Massimo Poncino, and Alessio Burrello. Lightweight software kernels and hardware extensions for efficient sparse deep neural networks on microcontrollers. 7:1–13, 2025.
- [4] Karol C. Jurzec, Tomasz Szydlo, and Maciej Wielgosz. Compression and inference of spiking neural networks on resource-constrained hardware. *arXiv preprint arXiv:2511.12136*, 2025.
- [5] Shiming Li, Luca Mottola, Yuan Yao, and Stefanos Kaxiras. Efficient cnn inference on ultra-low-power mcus via saturation-aware convolution. pages 1–3, 2026.
- [6] Benjamin Graham and Laurens van der Maaten. Submanifold sparse convolutional networks. *arXiv preprint arXiv:1706.01307*, 2017.
- [7] M. Yao, O. Richter, G. Zhao, et al. Spike-based dynamic computing with asynchronous sensing-computing neuromorphic chip. *Nature Communications*, 15:4464, 2024.
- [8] Lloyd Watts. Event-Driven Simulation of Networks of Spiking Neurons. In *Neural Information Processing Systems*, pages 927–934, 1993.
- [9] ARM Ltd. Cmsis-dsp software library. <https://github.com/ARM-software/CMSIS-DSP>.
- [10] Liangzhen Lai, Naveen Suda, and Vikas Chandra. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus, 2018.
- [11] ARM Ltd. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus. <https://github.com/ARM-software/CMSIS-NN>.

Example Code

Figure 5: Transformation function definitons

Figure 6: Pareto graph of different image transformation techniques